

Central de Distribuição de Pedidos – Resultados e Respostas

Nota: valores obtidos em execução real dos scripts. Como os vetores aleatórios mudam a cada execução, uma nova rodada produz números levemente diferentes, mas a mesma tendência geral.

Etapa 3 – Resultados do Experimento Principal

Tam.	Bub. Comp.	Bub. Trocas	Ins. Comp.	Ins. Mov.	Sel. Comp.	Sel. Trocas	Qck. Comp.	Qck. Mov.
10	45	25	33	25	45	7	25	25
20	190	103	120	103	190	15	63	63
1.000	499.500	241.671	242.662	241.671	499.500	989	10.678	10.678

Etapa 4 – Análise dos Resultados

a) Menor número de comparações com 10 elementos?

Quick Sort, com 25 comparações — Bubble e Selection empataram em 45, e Insertion ficou em 33.

b) Menor número de trocas/movimentações?

Selection Sort, com apenas 7 trocas. Ele só troca uma vez por passada externa, mesmo comparando bastante.

c) O comportamento se manteve semelhante com 20 elementos?

Sim — Quick Sort continuou com o menor número de comparações (63) e Selection Sort com o menor número de trocas (15). A proporção entre os algoritmos se manteve.

d) O que aconteceu com 1.000 elementos?

A diferença entre os algoritmos ficou enorme. Bubble e Selection dispararam para 499.500 comparações cada, Insertion ficou próximo (242.662), e o Quick Sort permaneceu bem abaixo, com 10.678.

e) Bubble, Insertion e Selection são todos $O(n^2)$ — tiveram exatamente a mesma quantidade de operações?

Não. Bubble e Selection tiveram exatamente o mesmo número de comparações (499.500 com 1.000 elementos), porque nenhum dos dois tem otimização para parar mais cedo. Já o Insertion Sort teve menos comparações (242.662), pois seu laço interno para de comparar assim que encontra a posição correta. Nas trocas, a diferença é ainda maior: Selection Sort fez só 989 trocas contra 241.671 do Bubble e do Insertion — mesma complexidade assintótica, mas comportamento bem diferente na prática.

f) Qual algoritmo apresentou maior crescimento no número de operações?

Em comparações, Bubble e Selection cresceram igualmente (e mais que o Quick Sort). Em trocas/movimentações, Bubble e Insertion tiveram o crescimento mais acentuado, saltando de ~25 para mais de 240 mil — enquanto o Selection Sort manteve suas trocas quase constantes (7 → 989), crescendo muito menos que os demais.

g) Como o Quick Sort se diferenciou dos demais?

Cresceu de forma muito mais lenta: de 25 para 10.678 comparações (multiplicou por ~430), enquanto os outros multiplicaram por mais de 10.000 vezes. Isso confirma na prática a vantagem do $O(n \log n)$ sobre o $O(n^2)$. (Ver também a análise do desafio adicional abaixo — essa vantagem depende muito da organização dos dados.)

h) Os resultados são coerentes com as complexidades teóricas?

Sim. Bubble, Insertion e Selection cresceram de forma compatível com $O(n^2)$, e o Quick Sort cresceu de forma muito mais lenta, compatível com $O(n \log n)$ no caso médio.

i) Qual algoritmo você escolheria para a central de distribuição?

Quick Sort, para volumes grandes de pedidos — é o único que se manteve rápido com 1.000 elementos. Ressalva importante: como mostra o desafio adicional, essa vantagem depende da escolha do pivô e da organização dos dados. Em produção, valeria usar uma estratégia de pivô mais robusta (ex: mediana de três) para evitar o pior caso.

Desafio Adicional (0,5) – Efeito da Organização Inicial dos Dados

Vetores de 1.000 elementos, em três organizações diferentes.

Vetor	Bub. Comp.	Bub. Trocas	Ins. Comp.	Ins. Mov.	Sel. Comp.	Sel. Trocas	Qck. Comp.	Qck. Mov.
Aleatório	499.500	256.860	257.854	256.860	499.500	992	11.637	11.637
Ordenado	499.500	0	999	0	499.500	0	499.500	499.500
Invertido	499.500	499.500	499.500	499.500	499.500	500	499.500	499.500

A organização inicial interfere na quantidade de operações de todos os algoritmos da mesma maneira?

Não, cada algoritmo reage de forma bem diferente:

Bubble Sort e Selection Sort: o número de comparações não muda (499.500 nos três casos), porque essas implementações sempre percorrem todos os pares possíveis, independente da ordem. O que muda são as trocas: zero no vetor já ordenado, máximo no invertido.

Insertion Sort: é o mais sensível à ordem dos dados, no sentido positivo. No vetor já ordenado, fez apenas 999 comparações e nenhuma movimentação — quase $O(n)$. No invertido, foi ao pior caso (499.500), precisando mover todo elemento para o início.

Quick Sort: é o mais sensível no sentido negativo. No vetor aleatório, se comportou muito bem (11.637 comparações). Mas nos vetores já ordenado e invertido, caiu no pior caso teórico (499.500 comparações — igual ao Bubble Sort), porque o pivô escolhido (o último elemento) deixa de dividir o problema de forma equilibrada. Essa implementação inclusive exigiu aumentar o limite de recursão do Python para não travar, evidenciando o quão desbalanceada fica a recursão nesses casos.

Conclusão: a organização inicial pode não afetar em nada (Bubble/Selection), favorecer enormemente (Insertion em dados ordenados) ou prejudicar drasticamente (Quick Sort em dados ordenados/invertidos) — não existe um padrão único.